

Revisão de ISA

Objetivos

Objetivo 1:

- Revisão panorâmica de ISA
- Para **aplicação** em programação de sistemas

Objetivo 2:

- Orientar estudantes sobre **cobertura**
- Mapeamento entre tópicos e exercícios

Sumário

1. Interface com compilador

- Tradutor:
 - Geração de **assembly** para ISA-alvo
- Otimizador:
 - Otimização do assembly para **desempenho**

1. Interface com montador e ligador

- Geração de binário **executável** para ISA-alvo

Interface c/ compilador (tradutor)

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	incondicionais	indiretos incondicionais			
If-then-else	beq, bne	j			p.90-91	
Laços	beq, bne	j			p.92-93	2.26
Switch/Case			jr	lw	Lab 2	
Procedimentos/Métodos		jal	jr, jral	lw, sw	p. 98-99, 101-102	2.31, 2.34
Semáforos	beq, bne			ll, sc	p. 122 e AEX8 (slides 32-33)	2.43

Exemplos e exercícios mostram como compilador gera:

- if-then-else (pelo menos um desvio condicional)
- laços (pelo menos um desvio condicional)

Interface c/ compilador (tradutor)

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos		indiretos			
	condicionais	incondicionais	incondicionais			
If-then-else	beq, bne	j			p.90-91	
Lacos	beq, bne	i			p.92-93	2.26
Switch/Case			jr	lw	Lab 2	
Procedimentos/Métodos		jai	jr, jrai	lw, sw	p. 98-99, 101-102	2.31, 2.34
Semáforos	beq, bne			ll, sc	p. 122 e AEX8 (slides 32-33)	2.43

O roteiro do Lab 2 mostra como compilador gera um switch/case:

- **lw** usada para obter um endereço-alvo de uma tabela e carregá-lo em registrador;
- **jr** usado para carregar PC com o endereço-alvo do caso a ser executado.

Interface c/ compilador (tradutor)

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos		indiretos			
	condicionais	incondicionais	incondicionais			
If-then-else	beq, bne	j			p.90-91	
Laços	beq, bne	j			p.92-93	2.26
Switch/Case			ir	lw	Lab 2	
Procedimentos/Métodos		jal	jr, jral	lw, sw	p. 98-99, 101-102	2.31, 2.34
Semáforos	beq, bne			ll, sc	p. 122 e AEX8 (slides 32-33)	2.43

Exemplos e exercícios mostram como compilador gera procedimentos:

- **jal** usada para chamar um procedimento (rotina chamada),
- **jr** usado retornar do procedimento (rotina chamadora).

Uma **convenção de chamada** é usada para organizar a geração e torná-la eficiente.

Procedimentos: rotina folha

Rotina chamadora

```
...  
jal leaf_example  
...
```

Divisão de tarefas entre
chamadora e chamada:

Chamadora supõe que \$t0-
\$t8 **não são preservados**
pela rotina chamada

Chamadora supõe que \$s0-
\$s7 **são preservados** pela
rotina chamada
(se usados)

Subrotina chamada

```
addi $sp, $sp, -4  
sw $t1, 8($sp)  
sw $t0, 4($sp)  
sw $s0, 0($sp)  
add $t0, $a0, $a1  
add $t1, $a2, $a3  
sub $s0, $t0, $t1  
add $v0, $s0, $zero  
lw $s0, 0($sp)  
lw $t0, 4($sp)  
lw $t1, 8($sp)  
addi $sp, $sp, 4  
jr $ra
```

A convenção de chamada **não** garante a preservação de \$t0-\$7, \$t8-\$t9: **se** a chamadora precisar dos valores nesses registradores **após** a chamada de leaf-example, **ela própria deverá preservá-los explicitamente.**

Convenção de chamada

Name	Register number	Usage	Preserved on call?
\$zero	0	The constant value 0	n.a.
\$v0-\$v1	2-3	Values for results and expression evaluation	no
\$a0-\$a3	4-7	Arguments	no
\$t0-\$t7	8-15	Temporaries	no
\$s0-\$s7	16-23	Saved	yes
\$t8-\$t9	24-25	More temporaries	no
\$gp	28	Global pointer	yes
\$sp	29	Stack pointer	yes
\$fp	30	Frame pointer	yes
\$ra	31	Return address	yes

Se o valor em um desses registradores **antes** da chamada de uma rotina precisar ser usado **após** a chamada, **ele deve ser preservado** pela rotina **chamadora**. Há duas formas: (1) salvar o valor em **\$s0-\$s7** (cuja preservação pela rotina chamada é garantida pela convenção de chamada); (2) salvar/restaurar diretamente em **pilha**.

Convenção de chamada

Name	Register number	Usage	Preserved on call?
\$zero	0	The constant value 0	n.a.
\$v0-\$v1	2-3	Values for results and expression evaluation	no
\$a0-\$a3	4-7	Arguments	no
\$t0-\$t7	8-15	Temporaries	no
\$s0-\$s7	16-23	Saved	yes
\$t8-\$t9	24-25	More temporaries	no
\$gp	28	Global pointer	yes
\$sp	29	Stack pointer	yes
\$fp	30	Frame pointer	yes
\$ra	31	Return address	yes

- Note que a rotina **chamada deve** preservar registradores \$s0-\$s8, mas apenas aqueles **cujos valores ela modificar**.

Convenção de chamada

Name	Register number	Usage	Preserved on call?
\$zero	0	The constant value 0	n.a.
\$v0-\$v1	2-3	Values for results and expression evaluation	no
\$a0-\$a3	4-7	Arguments	no
\$t0-\$t7	8-15	Temporaries	no
\$s0-\$s7	16-23	Saved	yes
\$t8-\$t9	24-25	More temporaries	no
\$gp	28	Global pointer	yes
\$sp	29	Stack pointer	yes
\$fp	30	Frame pointer	yes
\$ra	31	Return address	yes

- Note que a rotina **chamada deve** preservar o valor de \$ra.
- Mas quando a rotina chamada é **folha**, ela **não** precisa salvar/restaurar \$ra, pois as únicas instruções que modificam \$ra são jal e jalr.
- A preservação explícita **só** é necessária se houver rotina **aninhada**.

Exercício 2.31

2.31 [5] <§2.8> Implement the following C code in MIPS assembly. What is the total number of MIPS instructions needed to execute the function?

```
int fib(int n){  
    if (n==0)  
        return 0;  
    else if (n == 1)  
        return 1;  
    else  
        return fib(n-1) + fib(n-2);  
}
```

Este exercício revisa vários tipos de instruções inteiras: aritméticas, comparação, desvios diretos condicionais e incondicionais e desvios indiretos.

Exercício 2.31

2.31 [5] <§2.8> Implement the following C code in MIPS assembly. What is the total number of MIPS instructions needed to execute the function?

```
int fib(int n){  
    if (n==0)  
        return 0;  
    else if (n == 1)  
        return 1;  
    else  
        return fib(n-1) + fib(n-2);  
}
```

Há três casos em que a função retorna. Há duas opções de implementação: (1) **três** pontos de retorno, um para cada caso, (2) tratamento de cada caso seguido de desvio para um **único** ponto de retorno;

Exercício 2.31

2.31 [5] <§2.8> Implement the following C code in MIPS assembly. What is the total number of MIPS instructions needed to execute the function?

```
int fib(int n){  
    if (n==0)  
        return 0;  
    else if (n == 1)  
        return 1;  
    else  
        return fib(n-1) + fib(n-2);  
}
```

Exemplo de árvore de decisão: (supondo n em $\{0, 1, 2, 3, \dots\}$)

- teste0: se $n > 0$ desvie para teste1, senão retorna caso 0;
- teste1: se $n > 1$ desvie para gen, senão retorna caso 1;
- gen: retorna caso genérico.

Exercício 2.31

2.31 [5] <§2.8> Implement the following C code in MIPS assembly. What is the total number of MIPS instructions needed to execute the function?

```
int fib(int n){  
    if (n==0)  
        return 0;  
    else if (n == 1)  
        return 1;  
    else  
        return fib(n-1) + fib(n-2);  
}
```

Erros comuns: não preservar a0 e v0 entre chamadas sucessivas a fib.

Exercício 2.34

2.34 Translate function `f` into MIPS assembly language. If you need to use registers `$t0` through `$t7`, use the lower-numbered registers first. Assume the function declaration for `func` is “`int f(int a, int b);`”. The code for function `f` is as follows:

```
int f(int a, int b, int c, int d){  
    return func(func(a,b),c+d);  
}
```

- Na primeira chamada, `func` tem seus parâmetros (`a`, `b`) **já corretamente posicionados** em `$a0` e `$a1`.
- A convenção de chamada **não** garante que `func` preserve `$a2` e `$a3`.
- Logo, os valores em `$a2` e `$a3` precisam ser **preservados por `f`**, **antes** de invocar `func` pela **primeira** vez.
- Os valores em `$a2` e `$a3` precisam ser **restaurados** antes da **segunda** chamada a `func`.

Exercício 2.34

2.34 Translate function `f` into MIPS assembly language. If you need to use registers `$t0` through `$t7`, use the lower-numbered registers first. Assume the function declaration for `func` is “`int f(int a, int b);`”. The code for function `f` is as follows:

```
int f(int a, int b, int c, int d){  
    return func(func(a,b),c+d);  
}
```

Erros comuns:

- Não preservar `$a2` e `$a3` antes de invocar `func` pela **primeira** vez.
- Não copiar `$v0` em `$a0` antes de invocar `func` pela **segunda** vez.
- Não colocar o resultado de `c+d` em `$a1` antes de invocar `func` pela **segunda** vez

Interface c/ compilador (tradutor)

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos		indiretos			
	condicionais	incondicionais	incondicionais			
If-then-else	beq, bne	j			p.90-91	
Laços	beq, bne	j			p.92-93	2.26
Switch/Case			jr	lw	Lab 2	
Procedimentos/Métodos		jal	jr, jral	lw, sw	p. 98-99, 101-102	2.31, 2.34
Semáforos	beq, bne			ll, sc	p. 122 e AEX8 (slides 32-33)	2.43

Exemplos e exercícios mostram como compilador gera operações de sincronização para programas concorrentes:

- **ll e sc** usados para implementar **troca atômica**,
- **beq e bne** usados para verificar **semáforo livre e atomicidade**.

Exemplo da Seção 2.11, p. 122

```
again: addi $t0,$zero,1      ;copy locked value
      ll     $t1,0($s1)      ;load linked
      sc     $t0,0($s1)      ;store conditional
      beq    $t0,$zero,again ;branch if store fails
add     $s4,$zero,$t1      ;put load value in $s4
```

- Esta sequência executa uma **troca atômica** entre o valor em **\$t0** (1) e o valor atual do semáforo residente no endereço **0(\$s1)**.
- Mas **não** verifica se o semáforo está **livre ou não**.

Exemplo da Seção 2.11, p. 122

```
again: addi $t0,$zero,1      ;copy locked value
      ll     $t1,0($s1)      ;load linked
      sc     $t0,0($s1)      ;store conditional
      beq    $t0,$zero,again ;branch if store fails
add    $s4,$zero,$t1      ;put load value in $s4
```

- Erro comum: não perceber que esta sequência **só** realiza a **troca atômica**, mas **não** implementa um lock.
- Para implementar um lock, esta sequência deve ser completada para **testar se o semáforo está livre**.
- Isso requer uma **instrução extra** de desvio condicional.

Exemplo AEX8, slide 32

lock: la \$t0, sem

try: ll **\$t1**, **0(\$t0)**

Tenta iniciar operação atômica

bne \$t1, \$zero, try Se \$t1 = 1, semáforo ocupado: tenta de novo

addi \$t1, \$zero, 1 Semáforo livre, prepara para ocupá-lo

sc **\$t1**, **0(\$t0)** Tenta concluir operação atômica

beq \$t1, \$zero, try Se \$t1 = 0, não foi atômica: tenta de novo

/* capturou o semáforo */

jr \$ra

- Esta sequência alternativa **primeiramente verifica se o semáforo está livre** e, em caso positivo, realiza a **troca atômica**.
- Ela implementa uma função onde o semáforo **não** é passado como parâmetro: é fixo e definido pelo próprio procedimento.

Requisitos para implementar lock

Há várias alternativas para implementá-lo,

- Mas todas precisam de **dois testes**:
 - Um para verificar **se** o semáforo está **livre**;
 - Outro para verificar **se** a troca **atômica** teve sucesso.
- Se o semáforo for passado como parâmetro,
 - Ele **obviamente** deve ser passado conforme a convenção de chamada.

Exercício 2.43

2.43 [5] <§2.11> Write the MIPS assembly code to implement the following C code:

```
lock(lk);  
shvar=max(shvar,x);  
unlock(lk);
```

Assume that the address of the `lk` variable is in `$a0`, the address of the `shvar` variable is in `$a1`, and the value of variable `x` is in `$a2`. Your critical section should not contain any function calls. Use `ll/sc` instructions to implement the `lock()` operation, and the `unlock()` operation is simply an ordinary store instruction.

Este exercício supõe um **procedimento** que:

- Encapsula as 3 operações mostradas nas linhas acima (as demais linhas são omitidas);
- Foi invocado com 3 parâmetros: `$a0` (`lk`), `$a1` (`shvar`) e `$a2` (`x`);
- Escreve em variável compartilhada (`shvar`);
- Contém seção crítica que requer sincronização: `lock` e `unlock`.

Exercício 2.43

2.43 [5] <§2.11> Write the MIPS assembly code to implement the following C code:

```
lock(lk);  
shvar=max(shvar,x);  
unlock(lk);
```

Assume that the address of the `lk` variable is in `$a0`, the address of the `shvar` variable is in `$a1`, and the value of variable `x` is in `$a2`. Your critical section should **not contain any function calls.** Use `ll/sc` instructions to implement the `lock()` operation, and the `unlock()` operation is simply an ordinary store instruction.

- Este exercício supõe que `lock` e `unlock` **não** usam o mecanismo de chamada de rotinas **dentro** do procedimento.
- Portanto, as instruções para `lock` e `unlock` são expandidas localmente dentro do procedimento (i.e. **inline functions**).
- Assim, há 3 sequências complementares de instruções: uma para `lock`, outra para a atribuição e outra para `unlock` e **nenhuma instrução é jal.**

Exercício 2.43

2.43 [5] <§2.11> Write the MIPS assembly code to implement the following C code:

```
lock(lk);  
shvar=max(shvar,x);  
unlock(lk);
```

Assume that the address of the `lk` variable is in `$a0`, the address of the `shvar` variable is in `$a1`, and the value of variable `x` is in `$a2`. Your critical section should not contain any function calls. Use `ll/sc` instructions to implement the `lock()` operation, and the `unlock()` operation is simply an ordinary store instruction.

- Um novo valor deve ser atribuído a `shvar` **só** se `shvar < x`
- Se `shvar >= x`, não deve haver escrita em memória.
- Portanto, é necessário um teste e um desvio para **evitar escrita redundante**.

Exercício 2.43

2.43 [5] <§2.11> Write the MIPS assembly code to implement the following C code:

```
lock(lk);  
shvar=max(shvar,x);  
unlock(lk);
```

Assume that the address of the `lk` variable is in `$a0`, the address of the `shvar` variable is in `$a1`, and the value of variable `x` is in `$a2`. Your critical section should not contain any function calls. Use `ll/sc` instructions to implement the `lock()` operation, and the `unlock()` operation is simply an ordinary store instruction.

- Na prática `lock` e `unlock` costumam usar o **mecanismo de chamada de rotina**, ou seja:
 jal lock
 ... #instruções que implementam a atribuição
 ... #instruções que implementam a atribuição
 jal unlock

Interface c/ compilador (otimizador)

Tópico	ISA							Exemplos	
Seleção de instruções	SISD (MIPS)			aritmética inteira add, addu, addi, addiu, sub, subu, mult, multu, div, divu	acesso à memória lw, sw, lh, sh, lb, sb, ll, sc	lógica and, or, nor, andi, ori	comparação slt, slti, sltu, sltui	desvios beq, bne, j, jal, jr, jral	Seções 2.3, 2.6, 2.7, 2.8, 2.11
	SIMD (x86)	SSE		aritmética flutuante addss, addsd, mulss, mulsd	acesso à memória movss, movsd				Seções 3.7 e 3.8
			escalar	addps, addpd, mulps, mulpd	movps, movpd				
		AVX	escalar	vaddss, vaddsd, vmulss, vmulsd	vbroadcastss, vbroadcastsd				
			vetorial	vaddps, vaddpd, vmulps, vmulpd	vmovaps, vmovapd				
Alocação em registradores	SISD (MIPS)			inteiros e ponteiros s0-s7, t0-t7, t8-t9, a0-a4, v0-v1 (32 bits)					Figura 2.14
	SIMD (x86)	SSE		floats, doubles xmm0 a xmm15 (128 bits)					Figuras 3.20, 3.22, 3.24
				ymm0 a ymm15 (256 bits)					
			AVX						

Um compilador-otimizador seleciona instruções de forma a **reduzir o número de instruções executadas.**

Interface c/ compilador (otimizador)

Tópico	ISA							Exemplos
Seleção de instruções	SISD (MIPS)		aritmética inteira add, addu, addi, addiu, sub, subu, mult, multu, div, divu	acesso à memória lw, sw, lh, sh, lb, sb, ll, sc	lógica and, or, nor, andi, ori	comparação slt, slti, sltu, sltui	desvios beq, bne, j, jal, jr, jral	Seções 2.3, 2.6, 2.7, 2.8, 2.11
	SIMD (x86)	SSE	aritmética flutuante	acesso à memória				Seções 3.7 e 3.8
			escalar	addss, addsd, mulss, mulsd				
		vetorial	addps, addpd, mulps, mulpd	movps, movpd				
		AVX	escalar	vaddss, vaddsd, vmulss, vmulsd				
vetorial	vaddps, vaddpd, vmulps, vmulpd		vmovaps, vmovapd					
Alocação em registradores	SISD (MIPS)		inteiros e ponteiros s0-s7, t0-t7, t8-t9, a0-a4, v0-v1 (32 bits)					Figura 2.14
	SIMD (x86)	SSE	floats, doubles xmm0 a xmm15 (128 bits)					Figuras 3.20, 3.22, 3.24
		AVX	ymm0 a ymm15 (256 bits)					

Um compilador-otimizador seleciona instruções **SISD** (MIPS-like) para operações **inteiras** e instruções **SIMD** (AVX-like) para operações de **ponto flutuante**.

Interface c/ compilador (otimizador)

Tópico	ISA							Exemplos
Seleção de instruções	SISD (MIPS)		aritmética inteira add, addu, addi, addiu, sub, subu, mult, multu, div, divu	acesso à memória lw, sw, lh, sh, lb, sb, ll, sc	lógica and, or, nor, andi, ori	comparação slt, slti, sltu, sltui	desvios beq, bne, j, jal, jr, jral	Seções 2.3, 2.6, 2.7, 2.8, 2.11
	SIMD (x86)	SSE	aritmética flutuante addss, addsd, mulss, mulsd	acesso à memória movss, movsd				Seções 3.7 e 3.8
			addps, addpd, mulps, mulpd	movps, movpd				
		AVX	escalar vaddss, vaddsd, vmulss, vmulsd	vbroadcastss, vbroadcastsd				
			vetorial vaddps, vaddpd, vmulps, vmulpd	vmovaps, vmovapd				
	Alocação em registradores	SISD (MIPS)		inteiros e ponteiros s0-s7, t0-t7, t8-t9, a0-a4, v0-v1 (32 bits)				
SIMD (x86)		SSE	floats, doubles xmm0 a xmm15 (128 bits)					
			AVX	ymm0 a ymm15 (256 bits)				

Um compilador-otimizador seleciona instruções de forma a **reduzir o CPI médio**, através de exploração de dois tipos de **paralelismo**.

Interface c/ compilador (otimizador)

Tópico	ISA						Exemplos	
Seleção de instruções	SISD (MIPS)		aritmética inteira add, addu, addi, addiu, sub, subu, mult, multu, div, divu	acesso à memória lw, sw, lh, sh, lb, sb, ll, sc	lógica and, or, nor, andi, ori	comparação slt, slti, sltu, sltui	desvios beq, bne, j, jal, jr, jral	Seções 2.3, 2.6, 2.7, 2.8, 2.11
	SIMD (x86)	SSE	aritmética flutuante	acesso à memória				Seções 3.7 e 3.8
			escalar	addss, addsd, mulss, mulsd				
		vetorial	addps, addpd, mulps, mulpd	movps, movpd				
		AVX	escalar	vaddss, vaddsd, vmulss, vmulsd				
	vetorial		vaddps, vaddpd, vmulps, vmulpd	vmovaps, vmovapd				
Alocação em registradores	SISD (MIPS)		inteiros e ponteiros s0-s7, t0-t7, t8-t9, a0-a4, v0-v1 (32 bits)				Figura 2.14	
	SIMD (x86)	SSE	floats, doubles xmm0 a xmm15 (128 bits)				Figuras 3.20, 3.22, 3.24	
		AVX	ymm0 a ymm15 (256 bits)					

(1) *instruction-level parallelism*: compilador induz a execução paralela de múltiplas instruções SISD;

Interface c/ compilador (otimizador)

Tópico	ISA							Exemplos	
Seleção de instruções	SISD (MIPS)			aritmética inteira add, addu, addi, addiu, sub, subu, mult, multu, div, divu	acesso à memória lw, sw, lh, sh, lb, sb, ll, sc	lógica and, or, nor, andi, ori	comparação slt, slti, sltu, sltui	desvios beq, bne, j, jal, jr, jral	Seções 2.3, 2.6, 2.7, 2.8, 2.11
	SIMD (x86)	SSE	escalar	aritmética flutuante addss, addsd, mulss, mulsd	acesso à memória movss, movsd				Seções 3.7 e 3.8
			vetorial	addps, addpd, mulps, mulpd	movps, movpd				
		AVX	escalar	vaddss, vaddsd, vmulss, vmulsd	vbroadcastss, vbroadcastsd				
			vetorial	vaddps, vaddpd, vmulps, vmulpd	vmovaps, vmovapd				
Alocação em registradores	SISD (MIPS)			inteiros e ponteiros s0-s7, t0-t7, t8-t9, a0-a4, v0-v1 (32 bits)					Figura 2.14
	SIMD (x86)	SSE		floats, doubles xmm0 a xmm15 (128 bits)					Figuras 3.20, 3.22, 3.24
		AVX		ymm0 a ymm15 (256 bits)					

(2) *data-level parallelism*: compilador induz a execução paralela de **múltiplas operações** de uma única instrução SIMD.

Interface c/ compilador (otimizador)

Tópico	ISA							Exemplos	
Seleção de instruções	SISD (MIPS)			aritmética inteira add, addu, addi, addiu, sub, subu, mult, multu, div, divu	acesso à memória lw, sw, lh, sh, lb, sb, ll, sc	lógica and, or, nor, andi, ori	comparação slt, slti, sltu, sltui	desvios beq, bne, j, jal, jr, jral	Seções 2.3, 2.6, 2.7, 2.8, 2.11
	SIMD (x86)	SSE		aritmética flutuante addss, addsd, mulss, mulsd	acesso à memória movss, movsd				Seções 3.7 e 3.8
			escalar						
		AVX	vetorial	addps, addpd, mulps, mulpd	movps, movpd				
			escalar	vaddss, vaddsd, vmulss, vmulsd	vbroadcastss, vbroadcastsd				
vetorial	vaddps, vaddpd, vmulps, vmulpd	vmovaps, vmovapd							
Alocação em registradores	SISD (MIPS)			inteiros e ponteiros s0-s7, t0-t7, t8-t9, a0-a4, v0-v1 (32 bits)					Figura 2.14
	SIMD (x86)	SSE		floats, doubles xmm0 a xmm15 (128 bits)					Figuras 3.20, 3.22, 3.24
		AVX		ymm0 a ymm15 (256 bits)					

Um compilador-otimizador aloca **dados e ponteiros** em registradores para **reduzir o CPI médio e o número de instruções executadas**.

Interface c/ compilador (otimizador)

Tópico	ISA							Exemplos
Seleção de instruções	SISD (MIPS)		aritmética inteira add, addu, addi, addiu, sub, subu, mult, multu, div, divu	acesso à memória lw, sw, lh, sh, lb, sb, ll, sc	lógica and, or, nor, andi, ori	comparação slt, slti, sltu, sltui	desvios beq, bne, j, jal, jr, jral	Seções 2.3, 2.6, 2.7, 2.8, 2.11
	SIMD (x86)	SSE		aritmética flutuante addss, addsd, mulss, mulsd	acesso à memória movss, movsd			Seções 3.7 e 3.8
			escalar	addps, addpd, mulps, mulpd	movps, movpd			
		AVX	escalar	vaddss, vaddsd, vmulss, vmulsd	vbroadcastss, vbroadcastsd			
			vetorial	vaddps, vaddpd, vmulps, vmulpd	vmovaps, vmovapd			
	Alocação em registradores	SISD (MIPS)		inteiros e ponteiros s0-s7, t0-t7, t8-t9, a0-a4, v0-v1 (32 bits)				
SIMD (x86)		SSE	floats, doubles xmm0 a xmm15 (128 bits)					Figuras 3.20, 3.22, 3.24
		AVX	ymm0 a ymm15 (256 bits)					

Um compilador-otimizador aloca:

- Um **único** dado inteiro ou ponteiro em cada registrador SISD

Interface c/ compilador (otimizador)

Tópico	ISA							Exemplos	
Seleção de instruções	SISD (MIPS)			aritmética inteira add, addu, addi, addiu, sub, subu, mult, multu, div, divu	acesso à memória lw, sw, lh, sh, lb, sb, ll, sc	lógica and, or, nor, andi, ori	comparação slt, slti, sltu, sltui	desvios beq, bne, j, jal, jr, jral	Seções 2.3, 2.6, 2.7, 2.8, 2.11
	SIMD (x86)	SSE		aritmética flutuante addss, addsd, mulss, mulsd	acesso à memória movss, movsd				Seções 3.7 e 3.8
			escalar						
		AVX	vetorial	addps, addpd, mulps, mulpd	movps, movpd				
			escalar	vaddss, vaddsd, vmulss, vmulsd	vbroadcastss, vbroadcastsd				
		vetorial	vaddps, vaddpd, vmulps, vmulpd	vmovaps, vmovapd					
Alocação em registradores	SISD (MIPS)			inteiros e ponteiros s0-s7, t0-t7, t8-t9, a0-a4, v0-v1 (32 bits)				Figura 2.14	
	SIMD (x86)	SSE		floats, doubles				Figuras 3.20, 3.22, 3.24	
				xmm0 a xmm15 (128 bits)					
			AVX	ymm0 a ymm15 (256 bits)					

Um compilador-otimizador aloca:

- **Mútiplos** dados em ponto flutuante em cada registrador SIMD

Interface c/ montador/ligador

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	incondicionais	indiretos incondicionais			
instruções	beq, bne	j, jal	jr, jral	lw, sw		
formatos	I	J	R	I	p. 82, 83, 113	2.14, 2.15, 2.17, 2.18
alcance em bytes	$[-2^{17}, 2^{17})$	2^{28}	2^{32}	$[-2^{15}, 2^{15})$	p. 116	2.24, 2.40, 2.41, 2.42
modos de endereçamento	relativo ao PC	absoluto	absoluto via registrador	base = gp base ≠ gp	Fig. 2.18	2.39, 2.25
relocação automática?	sim	não	sim	não sim	AEX06 (slides 33-34)	

Montador e ligador geram código de acordo com o formato binário

- O montador define os campos opcode, funct, rs, rt, rd, shamt
- Montador e ligador definem campo address/offset dos formatos I e J

Interface c/ montador/ligador

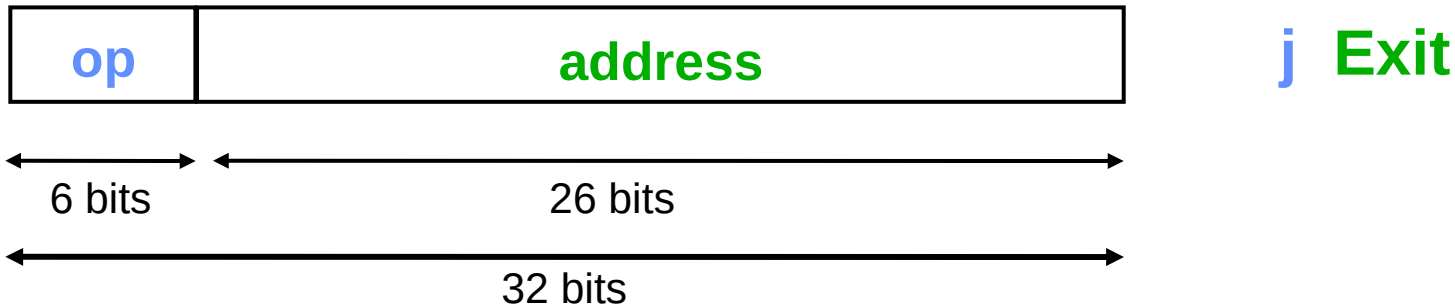
Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	incondicionais	indiretos incondicionais			
instruções	beq, bne	j, jal	jr, jral	lw, sw		
formatos	I	J	R	I	p. 82, 83, 113	2.14, 2.15, 2.17, 2.18
alcance em bytes	$[-2^{17}, 2^{17})$	2^{28}	2^{32}	$[-2^{15}, 2^{15})$	p. 116	2.24, 2.40, 2.41, 2.42
modos de endereçamento	relativo ao PC	absoluto	absoluto via registrador	base = gp base ≠gp	Fig. 2.18	2.39, 2.25
relocação automática?	sim	não	sim	não sim	AEX06 (slides 33-34)	

Montador e ligador verificam o alcance dos desvios

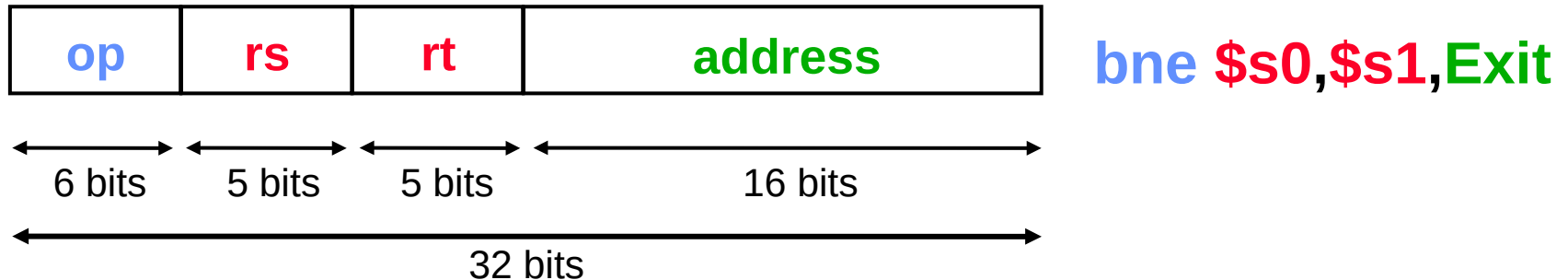
- Condicionais têm menor alcance
- Incondicionais têm maior alcance

Desvios: endereçamento

- Desvio incondicional: formato J



- Desvio condicional: formato I



Interface c/ montador/ligador

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	incondicionais	indiretos incondicionais			
instruções	beq, bne	j, jal	jr, jral	lw, sw		
formatos	I	J	R	I	p. 82, 83, 113	2.14, 2.15, 2.17, 2.18
alcance em bytes	$[-2^{17}, 2^{17})$	2^{28}	2^{32}	$[-2^{15}, 2^{15})$	p. 116	2.24, 2.40, 2.41, 2.42
modos de endereçamento	relativo ao PC	absoluto	absoluto via registrador	base = gp base ≠gp	Fig. 2.18	2.39, 2.25
relocação automática?	sim	não	sim	não sim	AEX06 (slides 33-34)	

Pode ser necessário substituir desvios dependendo do alcance requerido

Interface HW/SW

- Desvios condicionais
 - A maioria tem alvo próximo
 - Se deslocamento **não representável em 16 bits ?**
`beq $s0, $s1, L1`
...
- Solução: ajuste no montador ou ligador
 - Insere desvio incondicional para o alvo
 - Condição do desvio condicional é invertida
`bne $s0,$s1, L2`
`j L1`
L2: ...

Interface c/ montador/ligador

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	incondicionais	indiretos incondicionais			
instruções	beq, bne	j, jal	jr, jral	lw, sw		
formatos	I	J	R	I	p. 82, 83, 113	2.14, 2.15, 2.17, 2.18
alcance em bytes	$[-2^{17}, 2^{17}]$	2^{28}	2^{32}	$[-2^{15}, 2^{15}]$	p. 116	2.24, 2.40, 2.41, 2.42
modos de endereçamento	relativo ao PC	absoluto	absoluto via registrador	base = gp base ≠ gp	Fig. 2.18	2.39, 2.25
relocação automática?	sim	não	sim	não sim	AEX06 (slides 33-34)	

Campo de offset do formato I é usado por desvios condicionais e loads/stores, mas com **diferentes escalas**:

- Desvios condicionais: offset relativo ao PC codificado em palavras.
- Load/stores: offset em relação à base é codificado em bytes.

Interface c/ montador/ligador

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	incondicionais	indiretos incondicionais			
instruções	beq, bne	j, jal	jr, jral	lw, sw		
formatos	I	J	R	I	p. 82, 83, 113	2.14, 2.15, 2.17, 2.18
alcance em bytes	$[-2^{17}, 2^{17})$	2^{28}	2^{32}	$[-2^{15}, 2^{15})$	p. 116	2.24, 2.40, 2.41, 2.42
modos de endereçamento	relativo ao PC	absoluto	absoluto via registrador	base = gp base ≠ gp	Fig. 2.18	2.39, 2.25
relocação automática?	sim	não	sim	não sim	AEX06 (slides 33-34)	

Quando se reloca um código contendo um desvio condicional em **outra faixa de endereços**:

- O valor de PC onde reside o desvio varia automaticamente.
- O offset não se altera, porque a distância desvio-alvo é constante.
- Logo, o offset **não** precisa ser modificado após a relocação

Relocação automática

Endereço (0x)		Código	
-16 ↑	0000 0000	loop:	...
	0000 0004		...
	0000 0008		...
	0000 000C		bne \$s1, \$s2, loop

Endereço (0x)		Código	
-16 ↑	0FFF 0000	loop:	...
	0FFF 0004		...
	0FFF 0008		...
	0FFF 000C		bne \$s1, \$s2, loop

Interface c/ montador/ligador

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	incondicionais	indiretos incondicionais			
instruções	beq, bne	j, jal	jr, jral	lw, sw		
formatos	I	J	R	I	p. 82, 83, 113	2.14, 2.15, 2.17, 2.18
alcance em bytes	$[-2^{17}, 2^{17})$	2^{28}	2^{32}	$[-2^{15}, 2^{15})$	p. 116	2.24, 2.40, 2.41, 2.42
modos de endereçamento	relativo ao PC	absoluto	absoluto via registrador	base = gp base ≠ gp	Fig. 2.18	2.39, 2.25
relocação automática?	sim	não	sim	não sim	AEX06 (slides 33-34)	

Quando se reloca um código contendo um desvio direto incondicional em **outra faixa de endereços**:

- O endereço-alvo (absoluto) é modificado
- O campo address do formato J **precisa** ser modificado após a relocação

Interface c/ montador/ligador

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	incondicionais	indiretos incondicionais			
instruções	beq, bne	j, jal	jr, jral	lw, sw		
formatos	I	J	R	I	p. 82, 83, 113	2.14, 2.15, 2.17, 2.18
alcance em bytes	$[-2^{17}, 2^{17})$	2^{28}	2^{32}	$[-2^{15}, 2^{15})$	p. 116	2.24, 2.40, 2.41, 2.42
modos de endereçamento	relativo ao PC	absoluto	absoluto via registrador	base = gp base ≠ gp	Fig. 2.18	2.39, 2.25
relocação automática?	sim	não	sim	não sim	AEX06 (slides 33-34)	

Quando se reloca um código contendo um desvio indireto em **outra faixa de endereços**:

- Como o endereço-alvo (absoluto) não é especificado na instrução,
- A instrução não **precisa** ser modificada após a relocação.

Interface c/ montador/ligador

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	indiretos incondicionais	indiretos incondicionais			
instruções	beq, bne	j, jal	jr, jral	lw, sw		
formatos	I	J	R	I	p. 82, 83, 113	2.14, 2.15, 2.17, 2.18
alcance em bytes	$[-2^{17}, 2^{17})$	2^{28}	2^{32}	$[-2^{15}, 2^{15})$	p. 116	2.24, 2.40, 2.41, 2.42
modos de endereçamento	relativo ao PC	absoluto	absoluto via registrador	base = gp base ≠ gp	Fig. 2.18	2.39, 2.25
relocação automática?	sim	não	sim	não sim	AEX06 (slides 33-34)	

Quando se reloca o dado referenciado por uma instrução **load/store** nativa em **outra faixa de endereços**:

- Como o endereço-base (ponteiro) **varia** automaticamente,
- A instrução **não** precisa ser modificada após a relocação.

Interface c/ montador/ligador

Tópico	ISA				Exemplos	Exercícios
	desvios			load/store		
	diretos condicionais	indiretos incondicionais	indiretos incondicionais			
instruções	beq, bne	j, jal	jr, jral	lw, sw		
formatos	I	J	R	I	p. 82, 83, 113	2.14, 2.15, 2.17, 2.18
alcance em bytes	$[-2^{17}, 2^{17})$	2^{28}	2^{32}	$[-2^{15}, 2^{15})$	p. 116	2.24, 2.40, 2.41, 2.42
modos de endereçamento	relativo ao PC	absoluto	absoluto via registrador	base = gp base ≠ gp	Fig. 2.18	2.39, 2.25
relocação automática?	sim	não	sim	não sim	AEX06 (slides 33-34)	

Quando o dado referenciado por uma **pseudoinstrução load/store** em **outra faixa de endereços**:

- Como o endereço-base (gp) é **fixo** (variável global)
- A instrução **precisa** ser modificada após a relocação.

Conclusões

ISA é **pré-requisito** para outros tópicos

- **Desempenho** depende da ISA
- **Pipeline e emissão múltipla** dependem da ISA
- **Montador, ligador e DLL** dependem da ISA
- **Compilador** depende da ISA

ISA será reavaliada em provas posteriores

- Diretamente: **questões específicas** sobre ISA
- Indiretamente: como **pré-requisito** em questões envolvendo tópicos avançados

Recomendações

Revisar exemplos e exercícios sobre ISA para garantir 100% de cobertura:

- Geração de **assembly** para ISA-alvo
- Otimização do assembly para **desempenho**
 - Afeta número de instruções executadas
 - Afeta o CPI médio
- Geração de binário **executável** para ISA-alvo
 - Envolve **montador e ligador**